```python
# Feedforward Neural Network using Keras + TensorFlow on MNIST
import tensorflow as tf
import matplotlib.pyplot as plt
import tensorflow as tf

# 1. Load MNIST dataset
(x_train, y_train), (x_test, y_test) =
tf.keras.datasets.mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 3s 0us/step
```

```python
# 2. Preprocess data
x_train = x_train.reshape(-1, 784).astype("float32") / 255.0
x_test = x_test.reshape(-1, 784).astype("float32") / 255.0

# 3. Define Feedforward Neural Network
model = tf.keras.Sequential([
tf.keras.layers.Input(shape=(784,)),
tf.keras.layers.Dense(128, activation="relu"),
tf.keras.layers.Dense(64, activation="relu"),
tf.keras.layers.Dense(10, activation="softmax")
])

# 4. Compile the model
model.compile(
    optimizer=tf.keras.optimizers.SGD(learning_rate=0.01),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# 5. Train the model
history = model.fit(
    x_train, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)
```

```
Epoch 1/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 11s 6ms/step - accuracy: 0.8322 - loss:
0.6353 - val_accuracy: 0.9217 - val_loss: 0.2796
Epoch 2/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9146 - loss:
0.2978 - val_accuracy: 0.9350 - val_loss: 0.2241
Epoch 3/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9287 - loss:
0.2461 - val_accuracy: 0.9475 - val_loss: 0.1878
Epoch 4/10
```

```
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9386 - loss:
0.2121 - val_accuracy: 0.9533 - val_loss: 0.1643
Epoch 5/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9458 - loss:
0.1868 - val_accuracy: 0.9598 - val_loss: 0.1458
Epoch 6/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9507 - loss:
0.1676 - val_accuracy: 0.9630 - val_loss: 0.1322
Epoch 7/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9566 - loss:
0.1518 - val_accuracy: 0.9668 - val_loss: 0.1235
Epoch 8/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9606 - loss:
0.1386 - val_accuracy: 0.9700 - val_loss: 0.1164
Epoch 9/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9632 - loss:
0.1273 - val_accuracy: 0.9678 - val_loss: 0.1099
Epoch 10/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9663 - loss:
0.1175 - val_accuracy: 0.9697 - val_loss: 0.1086

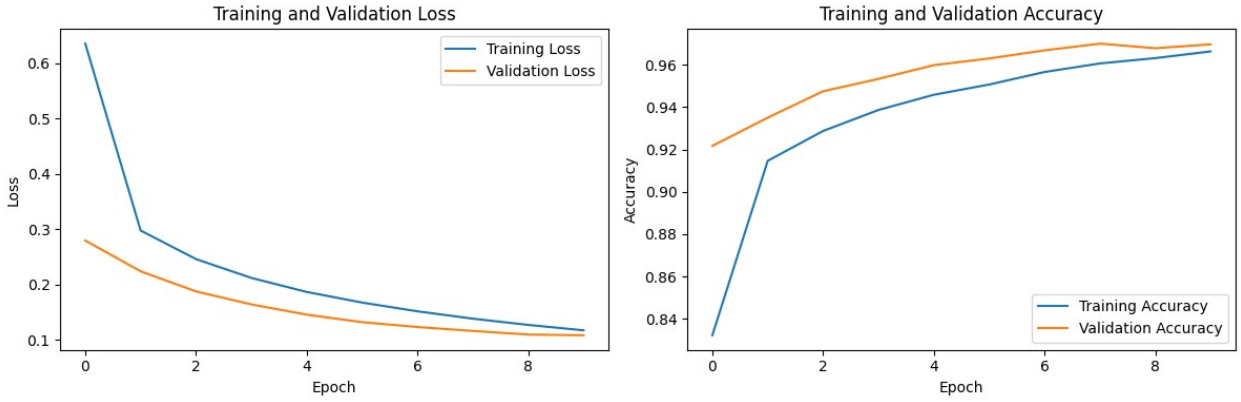
# 6. Evaluate the network
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print("Test Loss :", test_loss)
print("Test Accuracy :", test_accuracy)

Test Loss : 0.12293201684951782
Test Accuracy : 0.9617000222206116

# 7. Plot training loss and accuracy
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")
plt.legend()

# Training and validation accuracy
plt.subplot(1, 2, 2)
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy")
plt.legend()

plt.tight_layout()
plt.show()
```


Training and Validation Loss
Training Loss
Validation Loss
Loss
Epoch
Training and Validation Accuracy
Training Accuracy
Validation Accuracy
Accuracy
Epoch